

STUDY EDITION

DevOps Engineering Fieldbook

Volume 1

Linux Systems & Operations

Beginner-first operations textbook: problem, model, mechanism, experiment and evidence.

PURPOSE OF THIS CHAPTER

Understand the layer before choosing the command

Each chapter moves from a familiar operations problem to a visual mental model, mechanism, safe experiment, failure/debugging path, DevOps relevance, recall and lab.

HOW TO USE THIS BOOK

Read, run, observe, explain, rebuild

Use a disposable Ubuntu VM with snapshots. Record real output and keep raw evidence before changing state.

CORE mechanism TRY IT safe experiment TRAP misconception PRODUCTION operations
STOP & RECALL retrieval

THE STUDY LOOP



VOLUME 1 ROADMAP

Chapter 00 Linux mental model	Chapter 03 Search, Transform & Inspect	Next Processes, services, networking and recovery
---	--	---

Chapter 03 - Search, Transform & Inspect

3.1 Khi dữ liệu quá nhiều để đọc bằng mắt	3
3.2 Inspect trước khi search	3
3.3 head và tail	4
3.4 grep - tìm matching lines	4
3.5 Literal search trước regex	4
3.6 grep options quan trọng	5
3.7 wc - đếm evidence	5
3.8 sort và uniq	5
3.9 cut - lấy field đơn giản	6
3.10 find - tìm filesystem objects	6
3.11 find khác grep thế nào?	6
3.12 xargs - từ stdin sang command arguments	7
3.13 Pipe các tool nhỏ với nhau	8
3.14 Preview: awk và sed giải quyết bài toán gì?	8
3.15 Failure modes và debugging	9
3.16 STOP & RECALL	9
3.17 Guided Lab	9
3.18 Chapter Checkpoint	11

Search, Transform & Inspect

Name the layer, observe the state and test one hypothesis at a time.

CORE

HANDS-ON

PRODUCTION

3.1 Khi dữ liệu quá nhiều để đọc bằng mắt

Bạn có một file log nhỏ hôm qua. Hôm nay nó có hàng nghìn dòng. Câu hỏi thực tế là: **“Tôi có rất nhiều file/log lines. Làm sao tìm đúng thứ mình cần mà không đọc từng dòng bằng mắt?”**

Ta sẽ không bắt đầu bằng một command dài. Hãy tạo fixture disposable và biết trước nó chứa gì:

COMMAND

```
mkdir -p "$HOME/fieldbook-labs/ch03-search"
cd "$HOME/fieldbook-labs/ch03-search"
printf '%s\n' \
  '10:00 api INFO request-start' \
  '10:01 api ERROR timeout' \
  '10:02 worker ERROR queue-full' \
  '10:03 api INFO request-done' \
  '10:04 api ERROR timeout' \
  > events.log
```

Fixture có 5 lines, hai service (`api`, `worker`), hai `ERROR` lines cho `api` và một cho `worker`. Biết trước đáp án giúp ta kiểm tra tool thay vì tin mù quáng vào output.

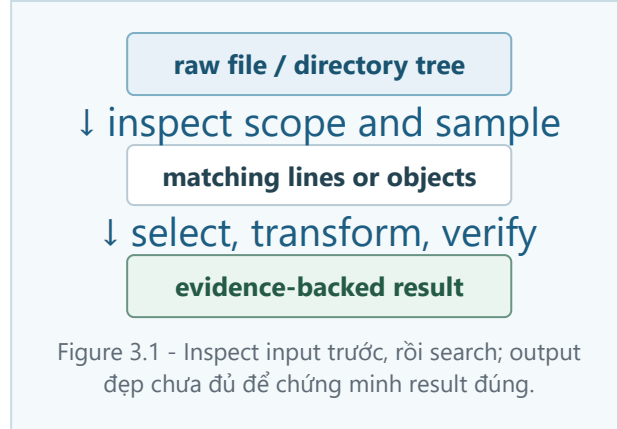
3.2 Inspect trước khi search

Trước khi search, xem file có thật và hiểu shape của nó:

COMMAND

```
pwd
ls -l events.log
wc -l events.log
head -n 3 events.log
tail -n 2 events.log
```

`ls` kiểm tra object/file name, `wc` đếm lines, còn `head` và `tail` cho sample ở đầu/cuối. Nếu `wc` nói 0 lines, đừng viết parser phức tạp; hãy kiểm tra input trước.



3.3 head và tail

Khi file dài, `head` xem phần đầu còn `tail` xem phần cuối:

COMMAND

```
head -n 2 events.log
tail -n 2 events.log
```

Đây là experiment về vị trí, không phải search. Trong DevOps, `tail -n 20 service.log` thường giúp xem những dòng mới nhất; nhưng hãy nhớ đó chỉ là một cửa sổ, không phải toàn bộ log.

3.4 grep - tìm matching lines

`grep` search **content**. Hãy tìm các dòng chứa text `ERROR`:

COMMAND

```
grep -F 'ERROR' events.log
```

`grep` đọc lines và in những lines match pattern. Với fixture, expected observation là 3 lines. Nếu muốn giữ raw evidence trong file mới, dùng `tee`:

COMMAND

```
grep -F 'ERROR' events.log | tee raw-errors.txt
```

`tee` vừa cho bạn xem output vừa ghi một bản copy; nó không sửa `events.log`. Nếu không có line nào, hãy kiểm tra spelling, path, và exit status ngay sau `grep`.

3.5 Literal search trước regex

Bắt đầu bằng **literal search**: tìm đúng chuỗi ký tự. `grep -F 'ERROR'` không diễn giải `.` hay `*` như regex.

COMMAND

```
printf '%s\n' 'v1.2 ready' 'v1x2 ready' > versions.txt
grep -F 'v1.2' versions.txt
grep -E 'v1.2' versions.txt
```

`grep -E` bật extended regular expression; dấu `.` lúc này có nghĩa “một ký tự bất kỳ”. Vì vậy hai command có thể match số dòng khác nhau. Khi chỉ cần tìm text cố định, `-F` làm ý định rõ hơn.

Đừng nhầm regex với shell glob. `*.log` là shell glob khi shell dùng nó để mở rộng tên file; nó không tự có cùng nghĩa bên trong một pattern regex. Regex thuộc tool như `grep -E`; glob thuộc shell.

3.6 grep options quan trọng

Một vài option đủ dùng cho beginner:

COMMAND

```
grep -n -F 'ERROR' events.log
grep -i -F 'error' events.log
grep -v -F 'INFO' events.log
```

`-n` thêm line number, `-i` bỏ phân biệt hoa/thường, `-v` lấy những dòng không match. Hãy thử từng option riêng để quan sát thay đổi. Quoting quan trọng: đặt pattern trong single quotes giúp shell không diễn giải các ký tự pattern trước khi `grep` nhận chúng.

3.7 wc - đếm evidence

Muốn biết có bao nhiêu matching lines, nối `grep` với `wc`:

COMMAND

```
grep -F 'ERROR' events.log | wc -l
```

Expected count là `3`. `wc -l events.log` đếm toàn file; `grep ... | wc -l` đếm output của `grep`. Hai câu hỏi khác nhau.

Nếu output là `0`, đó có thể là no match hợp lệ hoặc input/path sai. Dùng `grep -n -F 'ERROR' events.log` trước khi tin count. Exit status của `grep` và count do `wc` in ra là hai evidence khác nhau.

3.8 sort và uniq

`sort` sắp xếp lines. `uniq` gộp những lines giống nhau **liền kề**, vì thế thường sort trước:

COMMAND

```
grep -F 'ERROR' events.log | sort
grep -F 'ERROR' events.log | sort | uniq -c
```

Hãy nhìn kết quả thật của command thứ hai trước. Vì timestamp nằm trong mỗi dòng, ba dòng sau vẫn là ba **full lines** khác nhau:

TEXT EXAMPLE

```
1 10:01 api ERROR timeout
1 10:02 worker ERROR queue-full
1 10:04 api ERROR timeout
```

`uniq` chỉ so sánh toàn bộ line với line liền kề. Nó không biết rằng hai dòng `api` mô tả cùng một loại sự kiện sau khi bỏ timestamp. Muốn summary theo service, trước hết lấy field service ở vị trí thứ hai:

COMMAND

```
grep -F 'ERROR' events.log \  
| cut -d' ' -f2 \  
| sort \  
| uniq -c
```

Kết quả lúc này là:

TEXT EXAMPLE

```
2 api  
1 worker
```

[TRAP] `uniq -c` không hiểu **semantic event equivalence** (hai dòng có cùng ý nghĩa sự kiện) theo nghĩa của con người. Nó chỉ đếm các full lines giống nhau và liền kề. Hãy chọn field biểu diễn điều bạn muốn đếm, rồi mới `sort | uniq -c`.

Đừng dùng `uniq -c` trên input chưa sort rồi kết luận các dòng lặp đã được đếm hết.

3.9 cut - lấy field đơn giản

Các lines trong fixture có fields cách nhau bằng space: time, service, level, message. `cut` lấy phần đơn giản khi delimiter rõ:

COMMAND

```
cut -d' ' -f2 events.log  
grep -F 'ERROR' events.log | cut -d' ' -f2 | sort | uniq -c
```

`-d' '` chọn một space làm delimiter và `-f2` lấy field thứ hai. Đây là mô hình đơn giản, không phải parser cho mọi log. Nhiều spaces liên tiếp, tabs, hoặc message có format khác có thể làm assumption sai; hãy inspect input trước.

3.10 find - tìm filesystem objects

`find` search **filesystem objects/paths**, không search nội dung của từng line. Hãy tạo vài file an toàn rồi list:

COMMAND

```
mkdir -p logs/archive  
printf 'today\n' > logs/app.log  
printf 'old\n' > logs/archive/old.log  
printf 'temporary\n' > logs/debug.tmp  
find logs -type f -name '*.log' -print
```

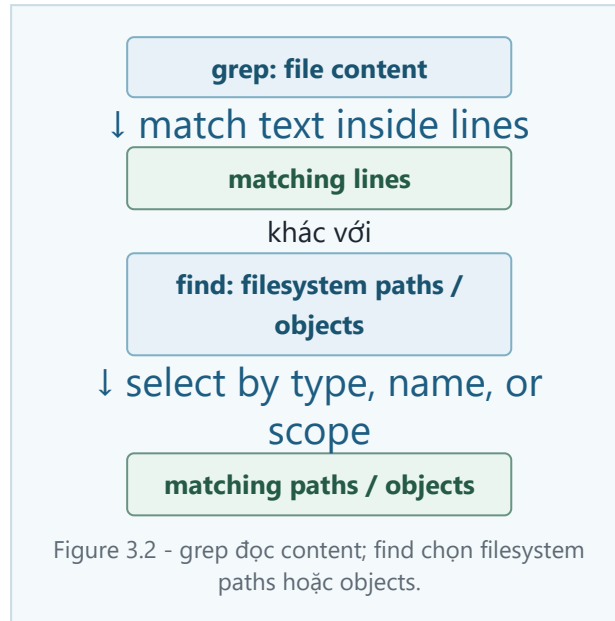
Expected paths là `logs/app.log` và `logs/archive/old.log`. `-type f` chọn regular files; `-name '*.log'` chọn filename theo shell-style pattern mà `find` tự xử lý; `-print` chỉ list, không mutate.

3.11 find khác grep thế nào?

Hãy nói thành câu trước khi chọn tool:

Câu hỏi	Tool	Nó tìm gì
Dòng nào chứa <code>ERROR</code> ?	<code>grep</code>	content trong text
File <code>.log</code> nào tồn tại?	<code>find</code>	filesystem objects/paths
Có bao nhiêu dòng match?	<code>wc</code> sau <code>grep</code>	output lines

`grep -R -F 'old' logs` tìm text content trong các file dưới `logs` ; nó không thay thế `find logs -name '*.log'` , vì một bên tìm **file content**, một bên duyệt **filesystem paths / objects** theo property. Khi cần cả hai, làm từng bước và kiểm tra output mỗi bước.

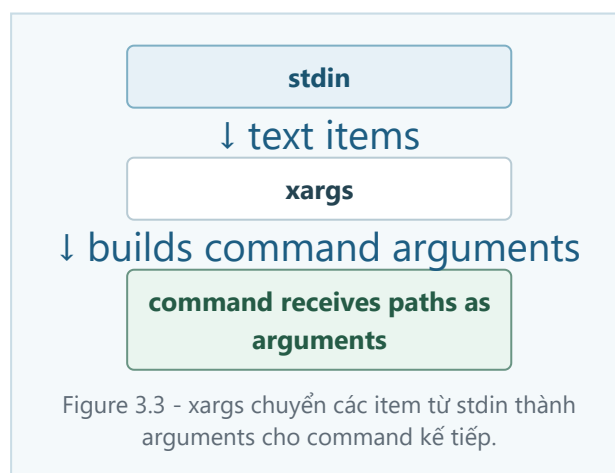


3.12 xargs - từ stdin sang command arguments

Có hai cách đưa dữ liệu tới command: **stdin** là stream text command đọc; **command arguments** là values nằm sau tên command trong command line. Ví dụ:

COMMAND
<pre>wc -l events.log cat < events.log</pre>

Lệnh đầu đưa filename như argument cho `wc` ; lệnh sau đưa bytes qua stdin. `xargs` đọc text từ stdin rồi biến các phần text đó thành arguments cho command khác.



Trước hết chỉ list:

COMMAND

```
find logs -type f -name '*.log' -print
```

Filename có space hoặc newline sẽ phá cách `xargs` mặc định chia text theo whitespace. Vì vậy không dạy pattern unsafe như production default. Sau khi hiểu problem, dùng delimiter NUL:

COMMAND

```
find logs -type f -name '*.log' -print0 | xargs -0 -r -n 1 wc -l --
```

`-print0` tạo separator an toàn hơn; `xargs -0` đọc separator đó; `-r` tránh chạy khi input rỗng; `-n 1` cho một path mỗi lần. `--` giúp path bắt đầu bằng `-` không bị hiểu là option. Đây là một command để inspect, không phải lời mời chạy mutation.

3.13 Pipe các tool nhỏ với nhau

Bây giờ ghép các tool đã biết:

COMMAND

```
grep -F 'ERROR' events.log | cut -d' ' -f2 | sort | uniq -c
```

Luồng suy nghĩ là: chọn matching lines → lấy service field → sort → count. Nếu summary sai, tháo pipeline ra và chạy từng đoạn. Đừng thêm command chỉ để làm output "đẹp" hơn.

3.14 Preview: awk và sed giải quyết bài toán gì?

`awk` phù hợp khi cần đọc fields và tính summary theo điều kiện; `sed` phù hợp khi cần inspect hoặc biến đổi từng line theo rule. Ở chapter này chỉ xem purpose, chưa học programming hay script phức tạp:

COMMAND

```
awk '$3 == "ERROR" { print $2 }' events.log  
sed -n '1,3p' events.log
```

`awk` in field 2 khi field 3 là `ERROR`; `sed -n '1,3p'` inspect ba dòng đầu. Nếu delimiter/schema đổi, cả hai command vẫn có thể chạy nhưng meaning sai. Phần Bash for DevOps sẽ đi sâu hơn; đừng dùng `sed -i` trên source hoặc production file ở lab này.

3.15 Failure modes và debugging

Symptom	Kiểm tra nhỏ nhất	Câu hỏi cần hỏi
<code>grep</code> không match	<code>pwd</code> , <code>wc -l</code> , <code>grep -n -F</code>	path/content có đúng không?
Count bằng 0	xem raw matching lines	no match hay input error?
<code>uniq -c</code> đếm thiếu	thêm <code>sort</code> trước <code>uniq</code>	duplicates đã liền kề chưa?
<code>cut</code> field sai	<code>head</code> và đếm delimiter	spaces/tabs có đúng assumption?
<code>find</code> không thấy file	<code>find logs -type f -print</code>	scope/type/name có đúng không?
<code>xargs</code> tách sai filename	tạo filename có space trong fixture	boundary stdin → arguments có an toàn?

Giữ raw input và raw matches trước transform. Với file rỗng, ghi rõ empty có nghĩa “không có evidence” hay “không có lỗi”. Với schema drift, test một dòng malformed và fail rõ thay vì in summary đáng tin giả.

3.16 STOP & RECALL

1. `grep` tìm content hay filesystem objects?
2. `find` tìm content hay paths/objects?
3. Vì sao literal search nên đi trước regex?
4. `stdin` khác command arguments thế nào?
5. Vì sao filename có whitespace nguy hiểm với `xargs` mặc định?
6. `find ... -print0 | xargs -0` giải quyết boundary nào?
7. Vì sao `uniq -c` thường cần `sort` trước?

MỘT CÂU ĐỂ TỰ CHẤM

Trước khi chạy pipeline, hãy nói input là text hay paths, tool đang chọn gì, delimiter là gì, và evidence nào sẽ chứng minh kết quả.

3.17 Guided Lab

Objective

Tạo một log fixture nhỏ, inspect trước khi search, giữ raw matches, tạo summary, tìm `.log` objects, và kiểm tra filename boundary mà không mutate source.

Safety boundary

Chỉ dùng `$HOME/fieldbook-labs/ch03-search`. Không `rm` , `mv` , `chmod` , `chown` , `sed -i` , hay `xargs` mutation trên production path. Mọi `find` bắt đầu bằng list phase.

Setup

COMMAND

```
mkdir -p "$HOME/fieldbook-labs/ch03-search/logs/archive"
cd "$HOME/fieldbook-labs/ch03-search"
printf '%s\n' '10:00 api INFO request-start' '10:01 api ERROR timeout' '10:02 worker ERROR queue-
full' '10:03 api INFO request-done' '10:04 api ERROR timeout' > events.log
printf 'today\n' > logs/app.log
printf 'old\n' > logs/archive/old.log
printf 'temporary\n' > logs/debug.tmp
```

Mission

1. Chạy `wc -l events.log`, `head -n 2 events.log`, và `tail -n 2 events.log`; ghi observations.
2. Chạy `grep -n -F 'ERROR' events.log | tee raw-errors.txt`; xác nhận có 3 matching lines.
3. Chạy `grep -F 'ERROR' events.log | cut -d' ' -f2 | sort | uniq -c`; kiểm tra `api` là 2 và `worker` là 1.
4. Chạy `find logs -type f -name '*.log' -print`; phân biệt paths với content.
5. Tạo filename có space bằng `printf 'x\n' > 'logs/file with space.log'`, rồi chạy `find logs -type f -name '*.log' -print0 | xargs -0 -r -n 1 wc -l --` để quan sát boundary an toàn.
6. Chạy `awk '$3 == "ERROR" { print $2 }' events.log` và `sed -n '1,3p' events.log` chỉ như preview; không dùng script nâng cao.

Expected observation

`events.log` có 5 lines; raw error output có 3 lines; summary có `api 2` và `worker 1`; `find` trả về ba `.log` paths sau khi tạo filename có space. Không file input nào bị sửa.

Learner observation

Evidence	Quan sát thật	Nó chứng minh	Nó chưa chứng minh
<code>head / tail</code>		sample ở boundary	toàn bộ file
<code>raw-errors.txt</code>		raw matching lines	parser đúng mọi schema
service summary		count theo field hiện tại	log format luôn ổn định
<code>find</code> output		paths/objects trong scope	nội dung file
<code>xargs -0</code> output		NUL-safe argument boundary	mutation an toàn

Reasoning questions

- Nếu `grep` trả no match, đó là empty result hay input error?
- Tại sao `grep -F 'ERROR'` và `find logs -name '*.log'` trả hai loại evidence khác nhau?
- Nếu thêm nhiều spaces giữa fields, `cut -d' ' -f2` còn đúng không?
- `stdin` của `xargs` và arguments của `wc` khác nhau ở đâu?
- Vì sao không dùng `find ... | xargs rm` làm default?

Cleanup

Kiểm tra `pwd`, sau đó chỉ xóa disposable lab directory khi đã lưu observations:

COMMAND

```
cd "$HOME/fieldbook-labs"  
rm -rf -- ch03-search
```

PASS criteria

- Raw fixture và raw matches được giữ trước transform.
- Summary khớp counts đã biết và có một sample kiểm tra thủ công.
- Bạn phân biệt được `grep` content với `find` objects/paths.
- Bạn giải thích được stdin versus arguments và whitespace risk của `xargs`.
- Không có destructive find action nào được chạy.

3.18 Chapter Checkpoint

Bạn đạt Chapter 03 khi có thể chọn đúng tool cho content search, object search, line inspection, counting, sorting, field extraction và argument handoff; đồng thời nói rõ glob khác regex, raw evidence cần được giữ, và output hợp lý chưa tự chứng minh schema hay parser đúng.

DevOps connection: Search/transform reasoning giúp bạn tìm ERROR lines, xem log tail, đếm status lặp, tìm `.log`` objects, lấy field nhỏ và điều tra evidence mà không phá input source.